

Introducción a Python para Economistas

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction	3
1 ¿Por qué es importante aprender un lenguaje de programación?	3
1.1 La transformación digital de la economía	3
1.2 Ventajas de programar como economista	3
1.3 Casos de uso en economía	3
2 Curva de aprendizaje y alcance de distintos lenguajes	4
2.1 Comparación de lenguajes para análisis económico	4
2.1.1 Stata	4
2.1.2 R	5
2.1.3 Python	5
2.1.4 C++	6
2.2 Tabla comparativa	6
2.3 Recomendación para economistas	6
3 ¿Qué es Data Science? ¿Qué es Machine Learning?	7
3.1 Data Science (Ciencia de Datos)	7
3.2 Machine Learning (Aprendizaje Automático)	7
3.3 Tipos de Machine Learning	8
3.3.1 Aprendizaje Supervisado (Supervised Learning)	8
3.3.2 Aprendizaje No Supervisado (Unsupervised Learning)	8

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

3.4	Aplicaciones de Machine Learning en economía	8
3.4.1	Caso 1: Predicción de morosidad bancaria	8
3.4.2	Caso 2: Detección de fraude fiscal	9
3.4.3	Caso 3: Predicción del tipo de cambio	9
3.5	La revolución del Big Data en economía	9
4	¿Por qué Python?	9
4.1	Python en el ecosistema de Data Science	9
4.2	Ventajas específicas de Python para economistas	10
4.2.1	1. Ecosistema completo para análisis económico	10
4.2.2	2. Automatización de reportes económicos	10
4.2.3	3. Web scraping para investigación económica	11
4.2.4	4. APIs para datos económicos	11
4.2.5	5. Reproducibilidad e investigación transparente	12
4.3	Python vs otros lenguajes: Decisión informada	12
4.4	Testimonios del mundo real	13
5	Preparando tu entorno de trabajo	13
5.1	Instalación de Python	13
5.1.1	Opción 1: Anaconda (Recomendada para principiantes)	13
5.1.2	Opción 2: Python.org (Instalación mínima)	13
5.2	Entornos de desarrollo	14
5.2.1	Jupyter Notebook (Recomendado para empezar)	14
5.2.2	Visual Studio Code (Para proyectos más grandes)	14
5.2.3	Google Colab (Opción en la nube, sin instalación)	15
5.3	Organización de proyectos	15
5.4	Primeros pasos en Jupyter	16
6	Tu primer programa en Python	17
6.1	Hola Mundo del economista	17
6.2	Calculadora básica	17
6.3	Variables y tipos de datos	17
6.4	Listas: Colecciones de datos	18
6.5	Diccionarios: Datos etiquetados	18
6.6	Tu primer análisis de datos con Pandas	19
6.7	Visualización básica	20
7	Ejercicios prácticos	20
7.1	Ejercicio 1: Calculadora de inflación acumulada	20
7.2	Ejercicio 2: Análisis de datos de exportaciones	21
7.3	Ejercicio 3: Función de utilidad	22
8	Conclusión	23
9	Publicaciones Similares	23

Introducción a Python para Economistas

1 ¿Por qué es importante aprender un lenguaje de programación?

1.1 La transformación digital de la economía

En las últimas décadas, la economía ha experimentado una revolución digital sin precedentes. Como economistas, nos enfrentamos diariamente a:

- **Grandes volúmenes de datos:** Series temporales macroeconómicas, microdatos de encuestas de hogares, datos de transacciones financieras, registros administrativos, entre otros.
- **Análisis cada vez más complejos:** Modelos econométricos avanzados, análisis de impacto, predicciones económicas, evaluación de políticas públicas.
- **Necesidad de reproducibilidad:** La investigación económica moderna exige que nuestros análisis sean replicables y transparentes.

1.2 Ventajas de programar como economista

Automatización de tareas repetitivas

Imagina que necesitas calcular el índice de Gini para los 25 departamentos del Perú utilizando la Encuesta Nacional de Hogares (ENAHOG). Sin programación, esto implicaría:

- Abrir cada archivo de datos manualmente
- Realizar cálculos en Excel repetidamente
- Alto riesgo de errores humanos
- Horas o días de trabajo

Con programación, este proceso se reduce a minutos y es 100% reproducible.

Análisis de datos más sofisticados

La programación te permite:

- Implementar modelos econométricos avanzados (VAR, GARCH, modelos de panel dinámico)
- Realizar simulaciones de Monte Carlo para análisis de riesgo
- Aplicar técnicas de machine learning para predicción económica
- Crear visualizaciones interactivas de datos económicos

Ventaja competitiva en el mercado laboral

Las instituciones que más demandan economistas hoy en día buscan profesionales con habilidades de programación:

- Banco Central de Reserva del Perú (BCRP)
- Ministerio de Economía y Finanzas (MEF)
- Instituciones financieras privadas
- Consultorías económicas
- Organismos internacionales (BID, Banco Mundial, FMI)

1.3 Casos de uso en economía

Ejemplo 1: Análisis macroeconómico

Un economista del BCRP necesita analizar la relación entre la tasa de interés y la inflación en los últimos 20 años. Con Python puede:

```
# Importar datos del BCRP
# Limpiar y transformar los datos
# Estimar modelos VAR
# Generar funciones impulso-respuesta
# Crear gráficos profesionales para reportes
```

Ejemplo 2: Evaluación de impacto

Un investigador evalúa el impacto de un programa social en los ingresos familiares:

```
# Cargar microdatos de encuestas
# Implementar matching propensity score
# Estimar diferencias en diferencias
# Calcular errores estándar robustos
# Generar tablas de resultados automáticas
```

Ejemplo 3: Predicción económica

Un analista financiero predice el tipo de cambio:

```
# Obtener datos históricos del tipo de cambio
# Aplicar modelos ARIMA y GARCH
# Comparar con modelos de machine learning
# Evaluar precisión de predicciones
# Automatizar reportes diarios
```

2 Curva de aprendizaje y alcance de distintos lenguajes

2.1 Comparación de lenguajes para análisis económico

2.1.1 Stata

Ventajas:

- Diseñado específicamente para econometría
- Sintaxis intuitiva para economistas
- Excelente documentación de métodos econométricos
- Ampliamente usado en investigación académica

Desventajas:

- Licencia costosa (aproximadamente \$1,200 USD para estudiantes)
- Limitado para tareas fuera de econometría tradicional
- Menos flexible para automatización compleja
- Comunidad más pequeña que Python o R

Curva de aprendizaje: Baja-Media (2-3 meses para dominar lo básico)

Ejemplo en Stata:

```
* Regresión simple
regress ingreso educacion experiencia

* Efectos fijos
xtreg ingreso educacion, fe
```

2.1.2 R

Ventajas:

- Gratuito y de código abierto
- Excelente para estadística y visualización de datos
- Gran cantidad de paquetes econométricos (plm, AER, forecast)
- RStudio como entorno amigable

Desventajas:

- Sintaxis puede ser inconsistente entre paquetes
- Menor rendimiento con grandes volúmenes de datos
- Menos usado fuera del ámbito académico/estadístico

Curva de aprendizaje: Media (3-4 meses para dominar lo básico)

Ejemplo en R:

```
# Regresión simple
modelo <- lm(ingreso ~ educacion + experiencia, data = datos)
summary(modelo)

# Visualización
ggplot(datos, aes(x = educacion, y = ingreso)) + geom_point()
```

2.1.3 Python

Ventajas:

- Gratuito y de código abierto
- Lenguaje de propósito general (útil más allá del análisis de datos)
- Ecosistema completo: análisis, machine learning, web scraping, automatización
- Gran demanda en el mercado laboral
- Comunidad masiva y en crecimiento
- Integración con otras tecnologías

Desventajas:

- Curva de aprendizaje inicial más pronunciada
- Requiere aprender varios paquetes (pandas, numpy, statsmodels)
- Menos paquetes econométricos especializados que Stata o R

Curva de aprendizaje: Media-Alta (4-6 meses para dominar lo básico)

Ejemplo en Python:

```
import pandas as pd
import statsmodels.formula.api as smf

# Regresión simple
modelo = smf.ols('ingreso ~ educacion + experiencia', data=datos).fit()
```

```
print(modelo.summary())

# Visualización
import matplotlib.pyplot as plt
plt.scatter(datos['educacion'], datos['ingreso'])
plt.show()
```

2.1.4 C++

Ventajas:

- Máximo rendimiento computacional
- Control total sobre recursos del sistema

Desventajas:

- Curva de aprendizaje muy alta
- Desarrollo lento
- No diseñado para análisis de datos
- Innecesario para la mayoría de aplicaciones en economía

Curva de aprendizaje: Muy Alta (12+ meses)

2.2 Tabla comparativa

Característica	Stata	R	Python	C++
Costo	Alto	Gratis	Gratis	Gratis
Curva de aprendizaje	Baja	Media	Media	Alta
Econometría	Excelente	Muy buena	Buena	Baja
Machine Learning	Básico	Muy buena	Excelente	Media
Visualización	Buena	Excelente	Muy buena	Baja
Automatización	Básica	Buena	Excelente	Muy buena
Velocidad de ejecución	Media	Baja	Media	Excelente
Demanda laboral	Media	Media	Muy alta	Alta
Tamaño de comunidad	Media	Grande	Muy grande	Grande

2.3 Recomendación para economistas

Si eres estudiante de pregrado:

- Comienza con **Python**. Te dará la mayor versatilidad y mejores oportunidades laborales.
- Aprende Stata si tu programa lo requiere (muchos cursos de econometría lo usan).

Si eres estudiante de posgrado en economía:

- **Python** o **R**, dependiendo de tu área de especialización.
- Macroeconomía/Finanzas: Python
- Microeconometría/Evaluación de impacto: R o Python
- Economía computacional: Python definitivamente

Si trabajas en el sector público:

- **Python** o **Stata**, dependiendo de la institución.
- MEF y BCRP están adoptando cada vez más Python.

3 ¿Qué es Data Science? ¿Qué es Machine Learning?

3.1 Data Science (Ciencia de Datos)

Definición:

Data Science es un campo interdisciplinario que utiliza métodos científicos, procesos, algoritmos y sistemas para extraer conocimiento e insights de datos estructurados y no estructurados.

En términos económicos:

Data Science es el conjunto de técnicas y herramientas que nos permiten:

1. **Recolectar** datos económicos de múltiples fuentes
2. **Limpiar y procesar** datos (a menudo el 80% del trabajo)
3. **Analizar** patrones y relaciones
4. **Visualizar** resultados de forma comprensible
5. **Comunicar** hallazgos para la toma de decisiones

Componentes del Data Science:

Data Science = Matemáticas + Estadística + Programación + Conocimiento del dominio

Para economistas:

- **Matemáticas:** Cálculo, álgebra lineal, optimización
- **Estadística:** Inferencia, pruebas de hipótesis, econometría
- **Programación:** Python, SQL, manejo de datos
- **Conocimiento del dominio:** Teoría económica, instituciones, contexto

3.2 Machine Learning (Aprendizaje Automático)

Definición:

Machine Learning es un subcampo de la inteligencia artificial que permite a las computadoras aprender patrones de los datos sin ser programadas explícitamente.

Diferencia con econometría tradicional:**Econometría tradicional:**

- Enfoque: Inferencia causal y prueba de teorías
- Objetivo: Entender **por qué** sucede algo
- Ejemplo: ¿Cuál es el efecto causal de la educación sobre los ingresos?
- Prioridad: Interpretabilidad y validez causal

Machine Learning:

- Enfoque: Predicción y reconocimiento de patrones
- Objetivo: Predecir **qué** va a suceder
- Ejemplo: ¿Cuál será el ingreso de una persona dado su perfil?
- Prioridad: Precisión predictiva

3.3 Tipos de Machine Learning

3.3.1 *Aprendizaje Supervisado (Supervised Learning)*

Aprendemos de datos etiquetados (conocemos la respuesta correcta).

Ejemplos:

1. **Clasificación:** Predecir si una empresa caerá en default (sí/no)
 - Variable objetivo: Binaria (default o no default)
 - Algoritmos: Regresión logística, árboles de decisión, random forests
2. **Regresión:** Predecir el precio de una vivienda
 - Variable objetivo: Continua (precio en soles)
 - Algoritmos: Regresión lineal, regresión ridge, redes neuronales

3.3.2 *Aprendizaje No Supervisado (Unsupervised Learning)*

Encontramos patrones en datos sin etiquetas.

Ejemplos:

1. **Clustering:** Segmentar consumidores en grupos similares
 - Identificar perfiles de ahorradores
 - Agrupar países por nivel de desarrollo
2. **Reducción de dimensionalidad:** Simplificar datos con muchas variables
 - Análisis de componentes principales (PCA)
 - Crear índices sintéticos

3.4 Aplicaciones de Machine Learning en economía

3.4.1 *Caso 1: Predicción de morosidad bancaria*

Un banco necesita predecir qué clientes tienen mayor probabilidad de no pagar su préstamo.

Enfoque tradicional (Regresión logística):

$P(\text{default}) = f(\text{ingresos}, \text{historial_crediticio}, \text{edad}, \text{educación})$

Enfoque ML (Random Forest):

- Considera interacciones complejas entre variables
- Puede capturar relaciones no lineales
- Mayor precisión predictiva (pero menos interpretable)

3.4.2 Caso 2: Detección de fraude fiscal

SUNAT quiere identificar empresas con alta probabilidad de evasión tributaria.

Variables:

- Ingresos declarados vs. estimados por sector
- Patrones inusuales en deducciones
- Relaciones con proveedores
- Historial de fiscalizaciones

Algoritmo: Anomaly detection o clasificación supervisada

3.4.3 Caso 3: Predicción del tipo de cambio

Modelos tradicionales:

- ARIMA: Serie temporal univariada
- VAR: Múltiples series, relaciones lineales

Modelos ML:

- Redes neuronales LSTM: Capturan patrones temporales complejos
- Ensemble methods: Combinan múltiples modelos para mejor predicción

3.5 La revolución del Big Data en economía

Nuevas fuentes de datos:

1. **Datos transaccionales:** Compras con tarjetas de crédito en tiempo real
2. **Web scraping:** Precios de e-commerce, anuncios de empleo
3. **Redes sociales:** Sentimiento económico, expectativas
4. **Imágenes satelitales:** Actividad económica nocturna, agricultura
5. **Datos de telefonía:** Movilidad, redes sociales

Ejemplo: Medición de pobreza con imágenes satelitales

Investigadores han usado machine learning para:

- Analizar imágenes satelitales de luz nocturna
- Predecir niveles de ingreso y pobreza
- Útil en países con escasas encuestas de hogares

Proceso: 1. Recolectar imágenes satelitales 2. Entrenar modelos de visión computacional 3. Predecir indicadores económicos 4. Validar con encuestas tradicionales

4 ¿Por qué Python?

4.1 Python en el ecosistema de Data Science

Python se ha convertido en el lenguaje dominante para Data Science y Machine Learning. Según encuestas recientes:

- 65% de científicos de datos usan Python como lenguaje principal
- 9 de las 10 principales empresas tecnológicas usan Python

4.2 Ventajas específicas de Python para economistas

4.2.1 1. Ecosistema completo para análisis económico

Librerías fundamentales:

- **NumPy**: Computación numérica, álgebra lineal
- **Pandas**: Manipulación de datos (como Excel pero programable)
- **Matplotlib/Seaborn**: Visualización de datos
- **Statsmodels**: Econometría tradicional
- **Scikit-learn**: Machine Learning
- **TensorFlow/PyTorch**: Deep Learning

Ejemplo de workflow típico:

```
import pandas as pd          # Cargar y limpiar datos
import matplotlib.pyplot as plt  # Visualizar
import statsmodels.api as sm   # Econometría
from sklearn.ensemble import RandomForestRegressor  # ML

# 1. Cargar datos
datos = pd.read_csv('enaho_2023.csv')

# 2. Limpiar y transformar
datos = datos.dropna()
datos['log_ingreso'] = np.log(datos['ingreso'])

# 3. Análisis estadístico
modelo_ols = sm.OLS(datos['log_ingreso'], datos[['educacion', 'experiencia']]).fit()
print(modelo_ols.summary())

# 4. Machine Learning
modelo_ml = RandomForestRegressor()
modelo_ml.fit(X_train, y_train)

# 5. Visualización
plt.scatter(datos['educacion'], datos['log_ingreso'])
plt.show()
```

4.2.2 2. Automatización de reportes económicos

Python te permite crear reportes automatizados que se actualizan con nuevos datos:

```
# Script que corre automáticamente cada mes
def generar_reporte_inflacion():
    # 1. Descargar datos del BCRP
    inflacion = descargar_datos_bcrp()

    # 2. Calcular estadísticas
    inflacion_mensual = calcular_inflacion_mensual(inflacion)
```

```

inflacion_anual = calcular_inflacion_anual(inflacion)

# 3. Crear gráficos
crear_graficos(inflacion_mensual, inflacion_anual)

# 4. Generar documento Word o PDF
crear_reporte_word(inflacion_mensual, inflacion_anual)

# 5. Enviar por email automáticamente
enviar_email_reporte()

```

4.2.3 3. Web scraping para investigación económica

Recolecta datos económicos de internet automáticamente:

```

import requests
from bs4 import BeautifulSoup

# Extraer precios de productos de supermercados online
def scrape_precios_supermercado(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.content, 'html.parser')

    productos = []
    for item in soup.find_all('div', class_='producto'):
        nombre = item.find('h3').text
        precio = float(item.find('span', class_='precio').text.replace('$', ''))
        productos.append({'nombre': nombre, 'precio': precio})

    return pd.DataFrame(productos)

# Crear tu propio índice de precios en tiempo real
precios_hoy = scrape_precios_supermercado('https://...')

```

4.2.4 4. APIs para datos económicos

Accede a bases de datos económicas programáticamente:

```

# Ejemplo: Datos del Banco Mundial
import pandas_datareader as pdr
from datetime import datetime

# PIB de Perú desde 2000
pib_peru = pdr.get_data_world_bank(
    'NY.GDP.MKTP.CD', # Código de PIB
    countries=['PER'],
    start=datetime(2000, 1, 1),
    end=datetime.now()
)

```

```
)

# Datos del BCRP
import requests
url_bcrp = 'https://estadisticas.bcrp.gob.pe/estadisticas/series/api/...'
response = requests.get(url_bcrp)
datos = response.json()
```

4.2.5 5. Reproducibilidad e investigación transparente

Todo tu análisis queda documentado en código:

```
"""
Análisis de determinantes del ingreso en Perú
Autor: Edison Achalma
Fecha: Enero 2026
Datos: ENAHO 2023

Este script replica el análisis del paper:
"Educación y retornos en el mercado laboral peruano"
"""

# El código es tu metodología explícita
# Cualquiera puede replicar tus resultados
# Cambios en datos o método son fáciles de implementar
```

4.3 Python vs otros lenguajes: Decisión informada

Elige Python si:

- Quieres máxima versatilidad (análisis + automatización + machine learning)
- Planeas trabajar en sector privado o startups
- Te interesa data science más allá de econometría
- Quieres habilidades transferibles a otros campos

Elige Stata si:

- Tu departamento/institución lo usa exclusivamente
- Haces principalmente econometría de panel y series de tiempo
- Necesitas comandos econométricos muy específicos
- Prefieres soluciones “out of the box”

Elige R si:

- Te enfocas en estadística y visualización académica
- Tu comunidad de investigación usa R
- Trabajarás principalmente en investigación académica

La mejor opción: Aprende Python + conocimientos básicos de Stata/R

- Python como lenguaje principal
- Stata/R para situaciones específicas
- Maximiza tu empleabilidad

4.4 Testimonios del mundo real

Banco Central de Reserva del Perú (BCRP):

- División de Modelamiento Macroeconómico usa Python para nowcasting
- Scripts de Python automatizan extracción de datos de Bloomberg
- Modelos DSGE implementados en Python

Ministerio de Economía y Finanzas (MEF):

- Uso creciente de Python para análisis de datos fiscales
- Automatización de reportes presupuestarios

Sector privado:

- Bancos: Scoring de crédito con machine learning en Python
- Consultoras: Análisis de mercado y predicciones en Python
- Fintech: Toda su infraestructura en Python

5 Preparando tu entorno de trabajo

5.1 Instalación de Python

5.1.1 Opción 1: Anaconda (Recomendada para principiantes)

Anaconda es una distribución de Python que incluye:

- Python
- Jupyter Notebook (entorno interactivo)
- Librerías principales pre-instaladas
- Gestor de paquetes (conda)

Pasos de instalación:

1. Visita anaconda.com/download
2. Descarga la versión para tu sistema operativo (Windows/Mac/Linux)
3. Ejecuta el instalador
4. Sigue las instrucciones (deja opciones por defecto)
5. Verifica la instalación:

```
# Abre terminal o Anaconda Prompt
python --version
# Debería mostrar: Python 3.11.x o similar
```

5.1.2 Opción 2: Python.org (Instalación mínima)

Si prefieres una instalación más ligera:

1. Visita python.org/downloads
2. Descarga Python 3.11 o superior
3. Durante instalación, marca “Add Python to PATH”
4. Instala paquetes individualmente:

```
pip install numpy pandas matplotlib jupyter statsmodels scikit-learn
```

5.2 Entornos de desarrollo

5.2.1 Jupyter Notebook (Recomendado para empezar)

Jupyter es un entorno interactivo ideal para análisis exploratorio y aprendizaje.

Iniciar Jupyter:

```
# Desde terminal o Anaconda Prompt
jupyter notebook
```

Esto abrirá tu navegador con una interfaz donde puedes:

- Crear notebooks (.ipynb)
- Combinar código, resultados y texto explicativo
- Ver gráficos en línea
- Exportar a PDF, HTML, etc.

Estructura de un notebook:

```
# Celda 1: Importar librerías
import pandas as pd
import numpy as np

# Celda 2: Cargar datos
datos = pd.read_csv('datos.csv')

# Celda 3: Análisis
print(datos.describe())

# Celda 4: Visualización
datos['ingreso'].hist()
```

5.2.2 Visual Studio Code (Para proyectos más grandes)

VS Code es un editor profesional con excelente soporte para Python.

Instalación:

1. Descarga desde code.visualstudio.com
2. Instala la extensión de Python
3. Configura tu intérprete de Python

Ventajas:

- IntelliSense (autocompletado inteligente)
- Depuración integrada
- Control de versiones con Git
- Extensiones para todo tipo de tareas

5.2.3 Google Colab (Opción en la nube, sin instalación)

Si no quieres instalar nada en tu computadora:

1. Visita colab.research.google.com
2. Inicia sesión con tu cuenta de Google
3. Crea un nuevo notebook

Ventajas:

- No requiere instalación
- GPU gratuita para cálculos pesados
- Fácil compartir con colaboradores

Desventajas:

- Requiere internet
- Sesiones limitadas en tiempo
- Menos control sobre el entorno

5.3 Organización de proyectos

Estructura recomendada:

mi_proyecto_economia/

```
datos/
  raw/                # Datos originales (NUNCA modificar)
    enaho_2023.csv
  processed/          # Datos procesados
    enaho_limpia.csv
  external/           # Datos externos
    indicadores_bcrp.csv

notebooks/            # Jupyter notebooks para exploración
  01_exploracion.ipynb
  02_limpieza.ipynb
  03_analisis.ipynb

src/                  # Scripts de Python
  data_cleaning.py
  analysis.py
  visualization.py

outputs/              # Resultados
  figuras/
    grafico_ingresos.png
  tablas/
    regresion_resultados.csv

requirements.txt      # Lista de paquetes necesarios
README.md             # Descripción del proyecto
```

Ejemplo de requirements.txt:

```
numpy==1.24.3
pandas==2.0.3
matplotlib==3.7.2
seaborn==0.12.2
statsmodels==0.14.0
scikit-learn==1.3.0
jupyter==1.0.0
```

Instalar dependencias:

```
pip install -r requirements.txt
```

5.4 Primeros pasos en Jupyter**Crear tu primer notebook:**

1. Abre Jupyter Notebook
2. Click en “New” → “Python 3”
3. Renombra el notebook: “mi_primer_analisis.ipynb”

Atajos útiles:

- **Shift + Enter:** Ejecutar celda y avanzar
- **Ctrl + Enter:** Ejecutar celda sin avanzar
- **A:** Insertar celda arriba
- **B:** Insertar celda abajo
- **D, D:** Eliminar celda
- **M:** Cambiar a celda de Markdown (texto)
- **Y:** Cambiar a celda de código

Celdas de Markdown para documentación:

```
# Análisis de Ingresos en Perú

# Objetivo
Analizar los determinantes del ingreso utilizando datos de la ENAHO 2023.

# Metodología
1. Cargar y limpiar datos
2. Análisis descriptivo
3. Regresión lineal múltiple
4. Interpretación de resultados

# Resultados preliminares
- Observaciones: 15,342
- Variable dependiente: Ingreso mensual (log)
```


6 Tu primer programa en Python

6.1 Hola Mundo del economista

Abre un nuevo notebook de Jupyter y escribe:

```
# Mi primer programa en Python
print("¡Hola, mundo económico!")
print("Estudiante de economía de la UNSCH")
```

Ejecuta con **Shift + Enter**. Deberías ver:

```
¡Hola, mundo económico!
Estudiante de economía de la UNSCH
```

6.2 Calculadora básica

Python puede funcionar como calculadora:

```
# Operaciones básicas
print(2 + 3)          # Suma: 5
print(10 - 4)         # Resta: 6
print(3 * 7)          # Multiplicación: 21
print(15 / 3)         # División: 5.0
print(2 ** 3)         # Potencia: 8
print(17 // 5)        # División entera: 3
print(17 % 5)         # Módulo (residuo): 2
```

Aplicación económica: Calcular IPC

```
# Calcular índice de precios al consumidor (IPC)
precio_canasta_hoy = 850.50
precio_canasta_base = 750.00

# IPC = (Precio actual / Precio base) * 100
ipc = (precio_canasta_hoy / precio_canasta_base) * 100

print(f"El IPC es: {ipc:.2f}")
# Salida: El IPC es: 113.40
```

6.3 Variables y tipos de datos

```
# Variables numéricas
pib_peru = 242_632 # PIB en millones de USD (guiones bajos para legibilidad)
poblacion = 33_715_471
tasa_inflacion = 3.25 # Porcentaje

# Variables de texto (strings)
pais = "Perú"
```

```
moneda = "Sol"

# Variables booleanas (True/False)
es_pais_emergente = True
esta_en_recesion = False

# Imprimir información
print(f"País: {pais}")
print(f"PIB: ${pib_peru:,} millones")
print(f"PIB per cápita: ${pib_peru / poblacion * 1_000_000:.2f}")
```

Salida:

País: Perú
PIB: \$242,632 millones
PIB per cápita: \$7,196.21

6.4 Listas: Colecciones de datos

```
# Lista de tasas de crecimiento del PIB (últimos 5 años)
crecimiento_pib = [2.4, 4.0, -11.0, 13.3, 2.7]

# Acceder a elementos (índices empiezan en 0)
print(f"Crecimiento en 2020 (pandemia): {crecimiento_pib[2]}%")

# Agregar nuevo dato
crecimiento_pib.append(3.5) # Proyección 2024

# Calcular promedio
promedio = sum(crecimiento_pib) / len(crecimiento_pib)
print(f"Crecimiento promedio: {promedio:.2f}%")
```

6.5 Diccionarios: Datos etiquetados

```
# Información económica de un país
economia_peru = {
    'pib': 242_632,
    'poblacion': 33_715_471,
    'inflacion': 3.25,
    'desempleo': 7.1,
    'moneda': 'PEN'
}

# Acceder a datos
print(f"Tasa de desempleo: {economia_peru['desempleo']}%")
print(f"Inflación: {economia_peru['inflacion']}%")
```

```
# Agregar nuevo indicador
economia_peru['reservas_internacionales'] = 74_500 # Millones USD

# Calcular PIB per cápita
pib_per_capita = economia_peru['pib'] / economia_peru['poblacion'] * 1_000_000
economia_peru['pib_per_capita'] = pib_per_capita

print(f"PIB per cápita: ${economia_peru['pib_per_capita']:.2f}")
```

6.6 Tu primer análisis de datos con Pandas

```
import pandas as pd

# Crear un DataFrame con datos de regiones de Perú
datos_regiones = {
    'region': ['Lima', 'Arequipa', 'Cusco', 'La Libertad', 'Piura'],
    'poblacion': [9_674_755, 1_497_438, 1_357_075, 2_016_771, 2_047_954],
    'pib_millones': [234_000, 18_500, 12_300, 24_100, 16_800]
}

df = pd.DataFrame(datos_regiones)

# Calcular PIB per cápita
df['pib_per_capita'] = df['pib_millones'] / df['poblacion'] * 1_000_000

# Mostrar resultados
print(df)

# Estadísticas descriptivas
print("\nEstadísticas descriptivas:")
print(df['pib_per_capita'].describe())

# Región con mayor PIB per cápita
region_mas_rica = df.loc[df['pib_per_capita'].idxmax(), 'region']
print(f"\nRegión con mayor PIB per cápita: {region_mas_rica}")
```

Salida:

	region	poblacion	pib_millones	pib_per_capita
0	Lima	9674755	234000	24192.831729
1	Arequipa	1497438	18500	12353.806853
2	Cusco	1357075	12300	9063.043478
3	La Libertad	2016771	24100	11949.742268
4	Piura	2047954	16800	8202.633428

```
Estadísticas descriptivas:
count          5.000000
```

```

mean      13152.411551
std       6406.398824
min       8202.633428
25%      9063.043478
50%     11949.742268
75%     12353.806853
max      24192.831729
Name: pib_per_capita, dtype: float64

```

Región con mayor PIB per cápita: Lima

6.7 Visualización básica

```

import matplotlib.pyplot as plt

# Crear gráfico de barras
plt.figure(figsize=(10, 6))
plt.bar(df['region'], df['pib_per_capita'], color='steelblue')
plt.xlabel('Región')
plt.ylabel('PIB per cápita (USD)')
plt.title('PIB per cápita por región en Perú')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

7 Ejercicios prácticos

7.1 Ejercicio 1: Calculadora de inflación acumulada

Crea un programa que calcule la inflación acumulada dados los índices de precios mensuales.

Datos:

```
ipc_mensual = [100.0, 100.5, 101.2, 101.8, 102.5, 103.0]
```

Tareas: 1. Calcula la inflación mensual (variación porcentual) 2. Calcula la inflación acumulada 3. Imprime los resultados

Solución esperada:

```
# Tu código aquí
```

```

ipc_mensual = [100.0, 100.5, 101.2, 101.8, 102.5, 103.0]

# Calcular inflación mensual
inflacion_mensual = []
for i in range(1, len(ipc_mensual)):
    inflacion = ((ipc_mensual[i] - ipc_mensual[i-1]) / ipc_mensual[i-1]) * 100

```

```

    inflacion_mensual.append(inflacion)
    print(f"Mes {i}: {inflacion:.2f}%")

# Inflación acumulada
inflacion_acumulada = ((ipc_mensual[-1] - ipc_mensual[0]) / ipc_mensual[0]) * 100
print(f"\nInflación acumulada: {inflacion_acumulada:.2f}%")

```

7.2 Ejercicio 2: Análisis de datos de exportaciones

Tienes datos de exportaciones de Perú por sectores.

```

exportaciones = {
    'sector': ['Minería', 'Agricultura', 'Pesca', 'Manufactura'],
    'valor_2022': [35_600, 7_200, 2_800, 12_400], # Millones USD
    'valor_2023': [38_100, 7_850, 2_950, 13_200]
}

```

Tareas:

1. Crea un DataFrame con estos datos
2. Calcula la variación porcentual por sector
3. Identifica el sector con mayor crecimiento
4. Calcula el total exportado en cada año

```

import pandas as pd

exportaciones = {
    'sector': ['Minería', 'Agricultura', 'Pesca', 'Manufactura'],
    'valor_2022': [35_600, 7_200, 2_800, 12_400],
    'valor_2023': [38_100, 7_850, 2_950, 13_200]
}

df_exp = pd.DataFrame(exportaciones)

# Calcular variación porcentual
df_exp['variacion_%'] = ((df_exp['valor_2023'] - df_exp['valor_2022']) / df_exp['valor_2022']) * 100

print("Exportaciones por sector:")
print(df_exp)

# Sector con mayor crecimiento
idx_max = df_exp['variacion_%'].idxmax()
print(f"\nSector con mayor crecimiento: {df_exp.loc[idx_max, 'sector']} ({df_exp.loc[idx_max, 'variacion_%']:.2f}%)")

# Total exportado
total_2022 = df_exp['valor_2022'].sum()
total_2023 = df_exp['valor_2023'].sum()
print(f"\nTotal exportado 2022: ${total_2022:,} millones")

```

```
print(f"Total exportado 2023: ${total_2023:,} millones")
print(f"Crecimiento total: {((total_2023 - total_2022) / total_2022) * 100:.2f}%")
```

7.3 Ejercicio 3: Función de utilidad

Crea una función que calcule la utilidad de un consumidor según la función Cobb-Douglas:

$$U(x, y) = x^{\alpha} * y^{(1-\alpha)}$$

Donde x e y son bienes consumidos y α es el parámetro de preferencia.

```
def utilidad_cobb_douglas(x, y, alpha):
    """
    Calcula la utilidad según función Cobb-Douglas

    Parámetros:
    x: cantidad del bien 1
    y: cantidad del bien 2
    alpha: parámetro de preferencia (0 < alpha < 1)

    Retorna:
    Utilidad total
    """
    # Tu código aquí
    pass

# Prueba la función
u = utilidad_cobb_douglas(x=10, y=20, alpha=0.5)
print(f"Utilidad: {u:.2f}")
```

```
def utilidad_cobb_douglas(x, y, alpha):
    """
    Calcula la utilidad según función Cobb-Douglas
    """
    utilidad = (x ** alpha) * (y ** (1 - alpha))
    return utilidad

# Prueba con diferentes combinaciones
print("Análisis de utilidad:")
print("-" * 40)

combinaciones = [
    (10, 20, 0.3),
    (10, 20, 0.5),
    (10, 20, 0.7),
    (20, 10, 0.5),
]
```

```
for x, y, alpha in combinaciones:
    u = utilidad_cobb_douglas(x, y, alpha)
    print(f"U({x}, {y}) con {alpha}: {u:.2f}")
```

8 Conclusión

Has completado la primera guía de Python para economistas. Has aprendido:

- Por qué es importante programar como economista
- Las diferencias entre lenguajes de programación
- Qué son Data Science y Machine Learning
- Por qué Python es ideal para economistas
- Cómo configurar tu entorno de trabajo
- Los fundamentos básicos de Python

En la siguiente guía aprenderás:

- Variables, expresiones y statements en profundidad
- Tipos de datos y conversiones
- Operadores y precedencia
- Entrada de datos del usuario
- Convenciones de nombres de variables

Recursos adicionales recomendados:

1. **Documentación oficial de Python:** docs.python.org
2. **Pandas para análisis de datos:** pandas.pydata.org
3. **Statsmodels para econometría:** statsmodels.org
4. **Kaggle Learn:** Tutoriales interactivos gratuitos

¡Nos vemos en la siguiente guía!

9 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Instalacion De Anaconda](#)
2.  [Configurar Entorno Virtual Python Anaconda](#)
3.  [01 Introducion A La Programacion Con Python](#)
4.  [02 Variables Expresiones Y Statements Con Python](#)
5.  [03 Objetos De Python](#)
6.  [04 Ejecucion Condicional Con Python](#)
7.  [05 Iteraciones Con Python](#)
8.  [06 Funciones Con Python](#)
9.  [07 Dataframes Con Python](#)
10.  [08 Prediccion Y Metrica De Performance Con Python](#)
11.  [09 Metodos De Machine Learning Para Clasificacion Con Python](#)
12.  [10 Metodos De Machine Learning Para Regresion Con Python](#)
13.  [11 Validacion Cruzada Y Composicion Del Modelo Con Python](#)
14.  [Visualizacion De Datos Con Python](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!